

Program 19:

```
vedsk0831@DESKTOP-83FU7HB:~$ cd ~/OSSP
mkdir -p Program19
cd Program19
nano program19.c
vedsk0831@DESKTOP-83FU7HB:~/OSSP/Program19$ gcc -Wall -Wextra program19.c -o program19
program19.c: In function 'handle_sigint':
program19.c:15:24: warning: unused parameter 'sig' [-Wunused-parameter]
   15 | void handle_sigint(int sig)
      |                      ^~~~~^~~~
program19.c: In function 'handle_sigterm':
program19.c:32:25: warning: unused parameter 'sig' [-Wunused-parameter]
   32 | void handle_sigterm(int sig)
      |                      ^~~~~^~~~
vedsk0831@DESKTOP-83FU7HB:~/OSSP/Program19$ ls -l
total 24
-rwxr-xr-x 1 vedsk0831 vedsk0831 16904 Oct  6 13:15 program19
-rw-r--r-- 1 vedsk0831 vedsk0831  3735 Oct  6 13:15 program19.c
vedsk0831@DESKTOP-83FU7HB:~/OSSP/Program19$ ./program19
=====
OSSP Program 19 - Signal Handling
=====
Commands:
  Any Linux command -> execute
  exit               -> exit shell

shell> sleep 30
^C
Shell: Forwarding SIGINT to foreground process
Process terminated by signal 2
shell> sleep 20
^C
Shell: Forwarding SIGINT to foreground process
Process terminated by signal 2
```

```
vedsk0831@DESKTOP-83FU7HB:~/OSSP/Program19$ ./program19
=====
OSSP Program 19 - Signal Handling
=====
Commands:
  Any Linux command -> execute
  exit               -> exit shell

shell> sleep 30
^C
Shell: Forwarding SIGINT to foreground process
Process terminated by signal 2
shell> sleep 20
^C
Shell: Forwarding SIGINT to foreground process
Process terminated by signal 2
shell> echo "Shell is alive"
"Shell is alive"
Process exited with status 0
shell> |
```

CODE:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <signal.h>
```

```
#include <sys/wait.h>
```

```
pid_t foreground_pid = -1;
```

```
/*
```

```
 * SIGINT handler
```

```
 * Ctrl+C sends SIGINT to the shell.
```

```
 * The shell forwards it to the foreground child.
```

```
*/
```

```
void handle_sigint(int sig)
```

```
{
```

```
    if (foreground_pid > 0) {
```

```
        printf("\nShell: Forwarding SIGINT to foreground process\n");
```

```
        kill(foreground_pid, SIGINT);
```

```
    } else {
```

```
        printf("\nShell: Ignoring SIGINT\n");
```

```
    }
```

```
}
```

```

/*
 * Generic signal handler
 */
void handle_sigterm(int sig)
{
    printf("\nShell received SIGTERM\n");
}

int main()
{
    char input[256];

    /*
     * Register SIGINT handler
     */
    struct sigaction sa;

    memset(&sa, 0, sizeof(sa));

    sa.sa_handler = handle_sigint;

    sigemptyset(&sa.sa_mask);

    sa.sa_flags = SA_RESTART;

    if (sigaction(SIGINT, &sa, NULL) == -1) {

        perror("sigaction SIGINT");
    }
}

```

```

    exit(1);
}

/*
 * Register SIGTERM handler
 */
struct sigaction term_action;

memset(&term_action, 0, sizeof(term_action));

term_action.sa_handler = handle_sigterm;

sigemptyset(&term_action.sa_mask);

term_action.sa_flags = SA_RESTART;

sigaction(SIGTERM, &term_action, NULL);

printf("=====\n");
printf(" OSSP Program 19 - Signal Handling\n");
printf("=====\n");

printf("Commands:\n");
printf(" Any Linux command -> execute\n");
printf(" exit          -> exit shell\n\n");

```

```
while (1) {

    printf("shell> ");
    fflush(stdout);

    if (fgets(input,
               sizeof(input),
               stdin) == NULL) {

        break;
    }

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0)
        continue;

    /*
     * Exit shell
     */
    if (strcmp(input, "exit") == 0)
        break;

    /*
     * Create child
```

```
*/  
  
pid_t pid = fork();  
  
  
if (pid < 0) {  
  
    perror("fork");  
    continue;  
}  
  
  
/*  
 * Child process  
 */  
if (pid == 0) {  
  
    /*  
     * Restore default SIGINT behavior  
     */  
    signal(SIGINT, SIG_DFL);  
  
    /*  
     * Restore default SIGTERM behavior  
     */  
    signal(SIGTERM, SIG_DFL);  
  
  
    char *args[64];
```

```
int argc = 0;
```

```
char *token = strtok(input, " ");
```

```
while (token != NULL &&
```

```
    argc < 63) {
```

```
    args[argc++] = token;
```

```
    token = strtok(NULL, " ");
```

```
}
```

```
args[argc] = NULL;
```

```
if (argc == 0)
```

```
    exit(0);
```

```
/*
```

```
 * Execute command
```

```
*/
```

```
execvp(args[0], args);
```

```
perror("execvp");

exit(1);
}

/*
 * Parent process
 */
else {

    /*
     * Store foreground process
     */
    foreground_pid = pid;

    /*
     * Wait for child
     */
    int status;

    waitpid(pid, &status, 0);

    /*
     * Clear foreground PID
     */
}
```

```
foreground_pid = -1;
```

```
if (WIFEXITED(status)) {
```

```
    printf("Process exited with status %d\n",
```

```
        WEXITSTATUS(status));
```

```
}
```

```
else if (WIFSIGNALED(status)) {
```

```
    printf("Process terminated by signal %d\n",
```

```
        WTERMSIG(status));
```

```
}
```

```
}
```

```
}
```

```
printf("\nShell terminated.\n");
```

```
return 0;
```

```
}
```